

Sistema Mecatrónico de Acceso Asistido mediante Reconocimiento de Voz, Control de Hardware a Bajo Nivel y Retroalimentación Auditiva.

Mijaíl André Agüero Cangalaya, José Antonio Casachahua Peña, Ccorahua Quispe Sthif, Alejandro Coronado Guiorzo

Universidad Nacional Federico Villarreal

RESUMEN

El presente artículo técnico describe el diseño arquitectónico y la implementación conceptual de un prototipo de acceso automatizado, destinado a restituir la autonomía de personas con limitaciones motrices y visuales severas. A diferencia de los sistemas comerciales domóticos basados en sensores de presencia genéricos, esta propuesta integra una interfaz de usuario fundamentada en el procesamiento de lenguaje natural y la retroalimentación acústica bidireccional. La arquitectura del sistema prescinde de abstracciones de software de alto nivel, centrándose en la explotación de los recursos de hardware del microcontrolador ESP32. Se detalla la orquestación del Acceso Directo a Memoria (DMA) para la manipulación de ráfagas de audio mediante el bus I2S, la gestión eficiente de la memoria SRAM para evitar desbordamientos, y la manipulación de registros de temporización para el control cinemático de actuadores. Este diseño evidencia una aplicación directa de los principios de Arquitectura del Computador para resolver una problemática crítica de accesibilidad universal.

I. INTRODUCCIÓN

La evolución de la tecnología de asistencia (Assistive Technology) ha permitido que poblaciones vulnerables recuperen grados significativos de independencia. Sin embargo, el desplazamiento autónomo a través de espacios físicos cerrados sigue siendo un desafío crítico.

Pacientes que padecen artritis reumatoide en etapas avanzadas, esclerosis lateral amiotrófica (ELA), o individuos con lesiones medulares con cuadriplejía, se encuentran físicamente imposibilitados de ejercer el torque necesario para manipular perillas mecánicas o empujar puertas convencionales.

En entornos hospitalarios o centros de rehabilitación, las soluciones estandarizadas suelen depender de sensores infrarrojos pasivos (PIR) o lectores de tarjetas RFID. Ambas tecnologías presentan deficiencias fundamentales en este contexto: los sistemas RFID requieren manipulación manual de una credencial, mientras que los sensores PIR carecen de "intencionalidad" (abren la puerta ante cualquier movimiento cruzado, comprometiendo la privacidad del paciente y la climatización de la sala).

Para solventar estas deficiencias, este proyecto propone el desarrollo de un sistema mecatrónico de acceso que valida la intención explícita del usuario mediante comandos de voz. Adicionalmente, el sistema aborda la problemática de los usuarios con discapacidades visuales concomitantes al integrar una respuesta auditiva confirmatoria ("Puerta abriendo"), eliminando la incertidumbre sobre el estado del entorno físico. A nivel académico, el desarrollo de esta solución se plantea como un ejercicio riguroso de diseño de sistemas embebidos, donde el control de los periféricos se realiza a un nivel cercano a la máquina, garantizando latencias mínimas y un comportamiento determinista.

II. OBJETIVOS

A. Objetivo General Diseñar, estructurar y validar una infraestructura mecatrónica de acceso asistido, evidenciando el control directo sobre los periféricos de hardware, los registros internos y la gestión de memoria de un sistema embebido de 32 bits, evitando la dependencia de librerías precompiladas de alto nivel.

B. Objetivos Específicos

1. Gestión de Datos de Alta Frecuencia: Implementar la captura y emisión de señales acústicas a través del bus digital estandarizado I2S, orquestando controladores de Acceso Directo a Memoria (DMA) para asegurar que la CPU principal no colapse durante el muestreo continuo del micrófono.
2. Control Cinemático a Nivel de Registros: Desarrollar un controlador de modulación por ancho de pulso (PWM) configurando directamente los temporizadores de hardware del microcontrolador, garantizando un desplazamiento del servomotor fluido y libre de interrupciones de red.
3. Auditoría de Desempeño Arquitectónico: Cuantificar instrumentalmente la latencia de respuesta de extremo a extremo, la fragmentación del *Heap* (memoria dinámica) y la estabilidad temporal de las señales, para validar la idoneidad del microcontrolador ante tareas de ejecución concurrente.

III. DIAGRAMA DE BLOQUES Y RUTA DE DATOS (DATAPATH)

El diseño del sistema no se concibe como un flujo lineal, sino como una arquitectura de procesamiento paralelo donde múltiples buses operan simultáneamente.

1. Capa de Adquisición Digital: Liderada por el micrófono omnidireccional INMP441. A diferencia de los micrófonos analógicos, este sensor digitaliza la onda sonora internamente y transmite una trama serializada estructurada. Esto elimina la necesidad de usar el conversor analógico-digital (ADC) del microcontrolador,

reduciendo el ruido eléctrico introducido por los motores.

2. Capa de Gestión de Memoria y Enrutamiento: El núcleo central (SoC ESP32) recibe los bits del micrófono. Mediante el controlador DMA, estos datos se escriben cíclicamente en bloques de la memoria RAM interna (*buffers*). La CPU solo interviene cuando un bloque está completamente lleno, tomando esos datos para procesar la intención del usuario.
3. Capa de Conectividad y NLP: A través del transceptor Wi-Fi integrado, el microcontrolador despacha los *buffers* de audio hacia una API alojada en la nube, la cual realiza el Procesamiento de Lenguaje Natural (NLP) y devuelve una bandera lógica de autorización.
4. Capa de Actuación y Retroalimentación: Tras la autorización, la CPU reescribe los registros del periférico MCPWM para modificar el ancho de pulso e iniciar la apertura mecánica a través del servomotor. Simultáneamente, el sistema lee un archivo de audio desde su memoria Flash y lo inyecta en el bus I2S de salida hacia el amplificador PAM8403, ejecutando la confirmación vocal.

IV. PLATAFORMA SELECCIONADA Y JUSTIFICACIÓN ARQUITECTÓNICA

La selección del System-on-a-Chip (SoC) ESP32 de Espressif Systems se sustenta en un análisis crítico frente a otras alternativas del mercado (como la familia Arduino AVR o los microprocesadores basados en Linux como Raspberry Pi).

- Superación del Cuello de Botella (Vs. Arduino): Arquitecturas de 8 bits como el ATmega328P (Arduino Uno) carecen de la memoria RAM (solo poseen 2 KB) y de la velocidad de reloj (16 MHz) necesarias para almacenar *buffers* de audio digital, haciendo físicamente imposible este proyecto. El ESP32, con sus 520 KB de SRAM y reloj de 240 MHz, soporta holgadamente esta carga de datos.

- **Determinismo de Tiempo Real (Vs. Raspberry Pi):** Aunque una Raspberry Pi posee inmensa potencia, opera sobre un sistema operativo complejo (Linux). Esto introduce latencias impredecibles causadas por el cambio de contexto (*context switching*) del planificador del sistema operativo, lo que puede provocar interrupciones bruscas en la señal PWM del motor ("tartamudeo mecánico"). El ESP32 permite una programación tipo *bare-metal* o asistida por un RTOS ligero (FreeRTOS), asegurando que los eventos críticos de hardware se atiendan en tiempos precisos de microsegundos.
- **Procesamiento Asimétrico (Dual-Core):** El ESP32 cuenta con dos núcleos independientes. El diseño arquitectónico de este proyecto asignará el mantenimiento de la pila TCP/IP (Wi-Fi) al "Núcleo 0", mientras que el procesamiento del audio por DMA y el cálculo de los temporizadores del motor se aislarán en el "Núcleo 1". Esta división garantiza que una caída en la conexión a internet no congele las operaciones mecánicas de seguridad de la puerta.

V. MÉTRICAS DE EVALUACIÓN INSTRUMENTAL

Para asegurar el rigor científico del proyecto durante las fases de prototipado, el rendimiento de la arquitectura será evaluado mediante las siguientes métricas físicas:

1. **Cronometría de Latencia de Extremo a Extremo (Tiempo de Respuesta):** Se medirá el intervalo temporal exacto desde el momento en que cesa la captura acústica (fin de la locución del comando de voz por parte del usuario) hasta el microsegundo en que se detecta el primer flanco de subida en el pin de control del servomotor. Esta medición evidenciará la eficiencia del código frente al retardo introducido por la red, buscando optimizar la respuesta para que el usuario no perciba un retraso antinatural en la apertura.
2. **Auditoría de Consumo de Memoria Volátil (SRAM y Heap):** El muestreo de audio a frecuencias estándar (ej. 16 kHz a 16 bits) consume rápidamente la memoria RAM. Se auditará constantemente la cantidad de memoria dinámica libre y el tamaño de los bloques contiguos más grandes disponibles. El objetivo de esta métrica es demostrar que los arreglos circulares de memoria configurados para el controlador DMA no generan "fugas de memoria" (*memory leaks*) ni desbordamientos (*stack overflows*) tras múltiples horas de operación continua.
3. **Análisis de Fluctuación de la Señal Mecatrónica (Jitter PWM):** El servomotor requiere recibir un pulso estricto cada 20 milisegundos (50Hz) para mantener su posición. Utilizando instrumentación de laboratorio (osciloscopio), se medirá el margen de error o desviación en la periodicidad de este pulso mientras el ESP32 transmite datos por Wi-Fi. Esta métrica probará la efectividad de usar temporizadores de hardware en lugar de simples bucles de software para el control de actuadores.

VI. PLAN DE CONTINUIDAD TECNOLÓGICA (IOT E IA EN EL BORDE)

El diseño del prototipo sienta las bases estructurales para escalar el sistema en asignaturas posteriores, orientándolo hacia tendencias de la industria 4.0:

- **Integración Telemétrica (IoT):** A corto plazo, el código fuente integrará un cliente MQTT ligero. El acceso dejará de ser aislado; cada apertura, intento fallido o bloqueo del motor publicará un mensaje hacia un servidor local (ej. Node-RED). Esto permitirá a la administración de un hospital visualizar un panel de control con el estado en vivo de todas las habitaciones, habilitando aperturas remotas de emergencia desde una estación de enfermería centralizada.
- **Implementación de Inteligencia Artificial Local (TinyML):** Actualmente, la transcripción de la voz requiere delegar el cálculo a una API en la nube. La máxima evolución de este proyecto consistirá en implementar un modelo de redes neuronales optimizado (TensorFlow Lite

for Microcontrollers) que resida directamente en la memoria Flash del ESP32. Al entrenar al microcontrolador para reconocer espectrogramas de audio específicos (la palabra clave), la dependencia de internet se anulará. Esto garantizará que la puerta asistida funcione instantáneamente, resguardando la privacidad del audio del paciente al no enviar datos fuera de la habitación.

VII. CRONOGRAMA DE EJECUCIÓN

El desarrollo seguirá un enfoque incremental alineado con las evaluaciones del curso:

- Semana 6 (E1 - Fase Actual): Estructuración de la propuesta arquitectónica, justificación de componentes frente a la problemática de accesibilidad y definición de métricas.
- Semana 7-9 (E2): Mapeo de la memoria del controlador, programación de la Máquina de Estados (FSM) a bajo nivel y primera demostración funcional demostrando el manejo de interrupciones para los periféricos I2S y PWM.
- Semana 10-12 (E3): Integración física en maqueta mecatrónica, calibración de sensores acústicos y recolección instrumental de latencias y consumo de RAM.
- Semana 13-14 (E4): Pruebas de estrés sistémico, optimización del firmware, presentación en Feria de Proyectos y entrega del primer manuscrito en formato IEEE.
- Semana 15 (E5): Levantamiento de observaciones, compilación del código fuente comentado, presentación técnica en aula y entrega final del artículo científico para nivel de congreso

VIII. REFERENCIAS

- Espressif Systems. (2024). *ESP32 technical reference manual* (Versión 5.1).

Espressif Systems.
https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

- OASIS. (2014). *MQTT Version 3.1.1* (OASIS Standard). Organization for the Advancement of Structured Information Standards. <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>
- Organización Mundial de la Salud. (2018, 18 de mayo). *Tecnología de asistencia*. Organización Mundial de la Salud. <https://www.who.int/es/news-room/fact-sheets/detail/assistive-technology>
- TensorFlow. (2023). *TensorFlow Lite for microcontrollers*. Google. <https://www.tensorflow.org/lite/microcontrollers>
- Warden, P., & Situnayake, D. (2019). *TinyML: Machine learning with TensorFlow Lite on Arduino and ultra-low-power microcontrollers*. O'Reilly Media. <https://www.oreilly.com/library/view/tinyml/9781492052036/>